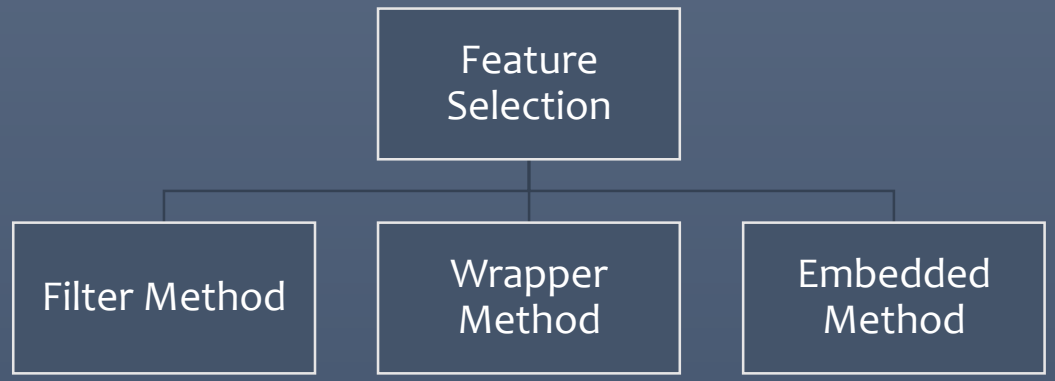
**Feature selection:**

Feature selection is a process in machine learning and statistical modelling where the goal is to identify and select a subset of relevant features from a larger set.

Below are the objective of the Feature Selection

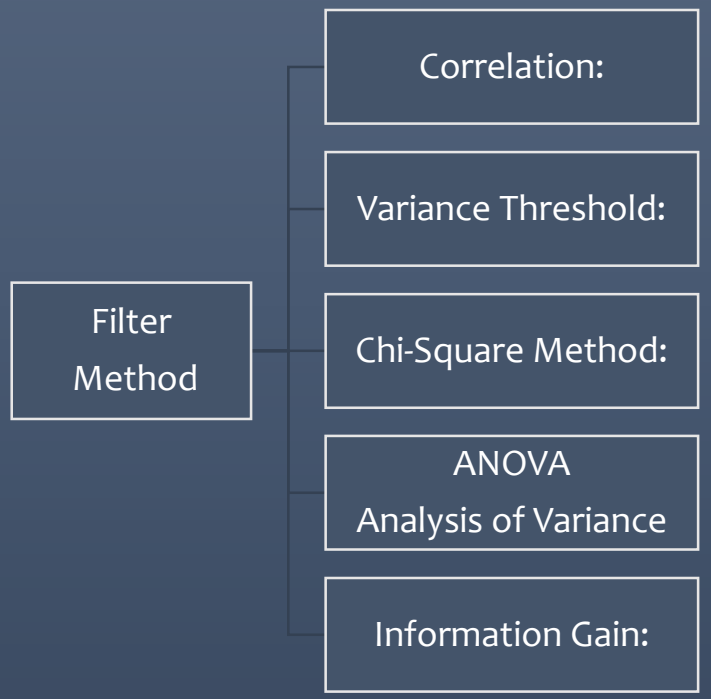
1) Improve the performance of a model,
2) Enhance its interpretability, and
3) Reduce computational costs.

Here's a breakdown of what feature selection involves and why it's important:



Filter Method:

Filter methods evaluate the relevance of features based on statistical properties of the data, independent of the machine learning algorithm. Features are selected or removed before model training, typically using univariate statistical tests or correlation measures. Some common techniques include:



Variance Threshold:

Variance thresholding is a simple method that removes features with low variance. Features with low variance contribute little information as they exhibit little variation across the dataset. By setting a threshold for variance, features below this threshold are considered uninformative and are removed from the dataset. This method is particularly useful for datasets with many features.

Ignore Target Variable:
Ignore Feature Interactions:
Sensitive to Data Scaling:
Arbitrary Threshold Value:

```
from sklearn.feature_selection import VarianceThreshold
selector = VarianceThreshold(threshold=0.5)
X_selected = selector.fit_transform(X)
```

**Correlation:**

Correlation measures the strength and direction of the linear relationship between two variables. In the context of feature selection, it helps identify features that are highly correlated with the target variable or with each other. High correlations between features indicate redundancy, and one of the correlated features can be removed to reduce multicollinearity. Correlation coefficients such as Pearson correlation coefficient or Spearman rank correlation coefficient are commonly used for this purpose.

```
corr_matrix = df.corr()
print("Correlation Matrix:")
print(corr_matrix)
```



**Chi-Square Method:**

The Chi-Square method is specifically used for feature selection in **classification tasks with categorical variables.** It measures the dependency between each categorical feature and the target variable by comparing the observed frequencies of feature-target pairs to the expected frequencies under the assumption of independence. Features with **higher chi-square scores are considered more relevant to the target variable.**

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
selector = SelectKBest(score_func=chi2, k=2)
X_selected = selector.fit_transform(X, y)
```

**ANOVA (Analysis of Variance):**

Evaluate each feature individually by calculating its relationship with the target variable using the p-value. If the p-value is above a specified threshold, keep the feature; otherwise, discard it.

There are two ways to calculate the Anova

1) when input is numerical and output is categorical value.
2) when input is numerical and output is numerical value.

**How to Calculate the p-Value:**

1) Calculate the F-Statistic:
2) Use the F-statistic to find the corresponding p-value.

**Calculate the F-Statistic:** Use the data to compute the F-statistic using below formula

$$\text{F-Statistics} = \frac{\text{Mean Sum of Squares Between Groups (MSB)}}{\text{Mean Sum of Squares Within Groups (MSW)}}$$

$$\text{MSB} = \frac{\text{SSB}}{\text{Degrees of Freedom Between Groups}}$$

$$\text{MSW} = \frac{\text{SSW}}{\text{Degrees of Freedom Within Groups}}$$

1. **Determine the p-Value:** Use the F-statistic to find the corresponding p-value.

```
from sklearn.feature_selection import f_classif
from sklearn.feature_selection import SelectKBest
import numpy as np

k = 5
f_scores, p_values = f_classif(X, y)
selector = SelectKBest(f_classif, k=k)
selected_features = selector.fit_transform(X, y)
selected_indices = np.argsort(-selector.scores_)[:k]

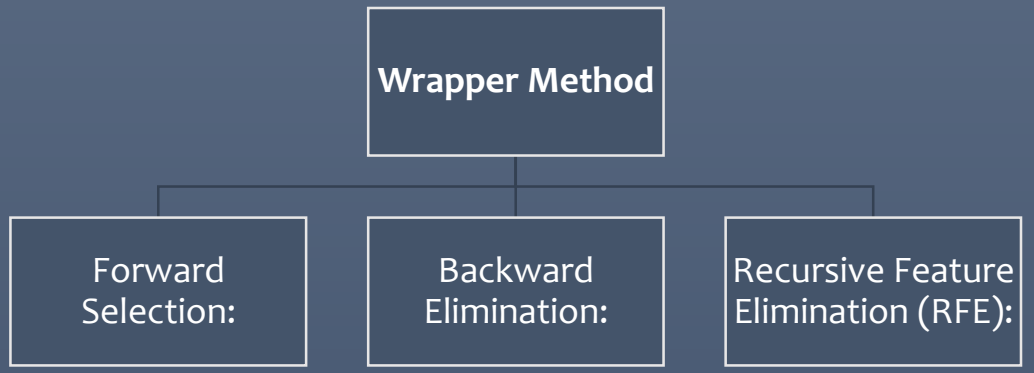
print(selected_indices)
# Print the selected feature names or columns
selected_feature_names = [feature_names[i] for i in selected_indices]
print(selected_feature_names)
```

**Information Gain:**

Information Gain, also known as mutual information, measures the amount of information gained about the target variable by knowing the value of a feature. It is calculated based on the entropy or Gini Impurity of the target variable before and after considering the feature. Features with higher information gain are considered more informative and are selected for model building.

```
import numpy as np
from sklearn.feature_selection import mutual_info_classif
feature = [['fea_01', 'fea_02', 'fea_03']]
target = [['region']]
mutual_info_classif(feature,target,random_state=0)
```

**Wrapper Method:**
Wrapper methods select subsets of features by repeatedly training and evaluating a model using different feature combinations. These methods consider the predictive performance of the model as the criterion for feature selection. Common wrapper methods include:

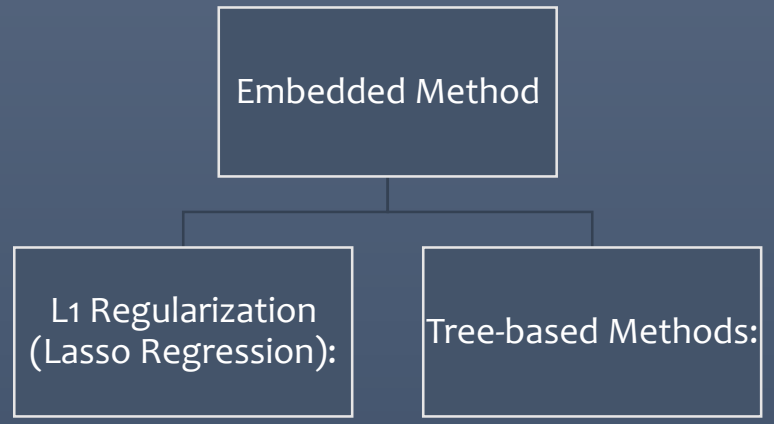



**Forward Selection:** Features are added incrementally, starting with an empty set and iteratively selecting the feature that improves model performance the most.

**Backward Elimination:** All features are initially included, and the least significant feature is removed iteratively until the desired number of features is reached.

**Recursive Feature Elimination (RFE):** RFE recursively removes the least important features based on a model's coefficient weights or feature importances until the desired number of features is reached.
Wrapper methods consider feature interactions and are more effective than filter methods for complex datasets. However, they can be computationally intensive, especially with a large number of features.

**Embedded Method:**
Embedded methods incorporate feature selection as part of the model training process. These methods leverage the intrinsic properties of certain machine learning algorithms to perform feature selection automatically. Common embedded methods include:




**L1 Regularization (Lasso Regression):** L1 regularization adds a penalty term based on the absolute value of the coefficients to the model's cost function. This encourages sparsity in the coefficients, effectively performing feature selection by driving some coefficients to zero.

**Tree-based Methods:** Decision trees and ensemble methods (e.g., Random Forest, Gradient Boosting Machines) inherently perform feature selection by evaluating feature importance during training. Features with higher importance scores are retained for the final model.


Feature Selection
Filter Method
Wrapper Method
Embedded Method
Correlation:
Forward Selection:
L1 Regularization (Lasso Regression):
Variance Threshold:
Backward Elimination:
Tree-based Methods:
Chi-Square Method:
Recursive Feature Elimination (RFE):
ANOVA (Analysis of Variance):
Information Gain: